

Evidencia Programación Orientada a Objetos con PHP

Aprendiz:

Jair Alfonso Arias Cueca

Instructor:

Ing. Néstor Montaña

Tecnólogo Análisis y Desarrollo de Software

Ficha: 3064241

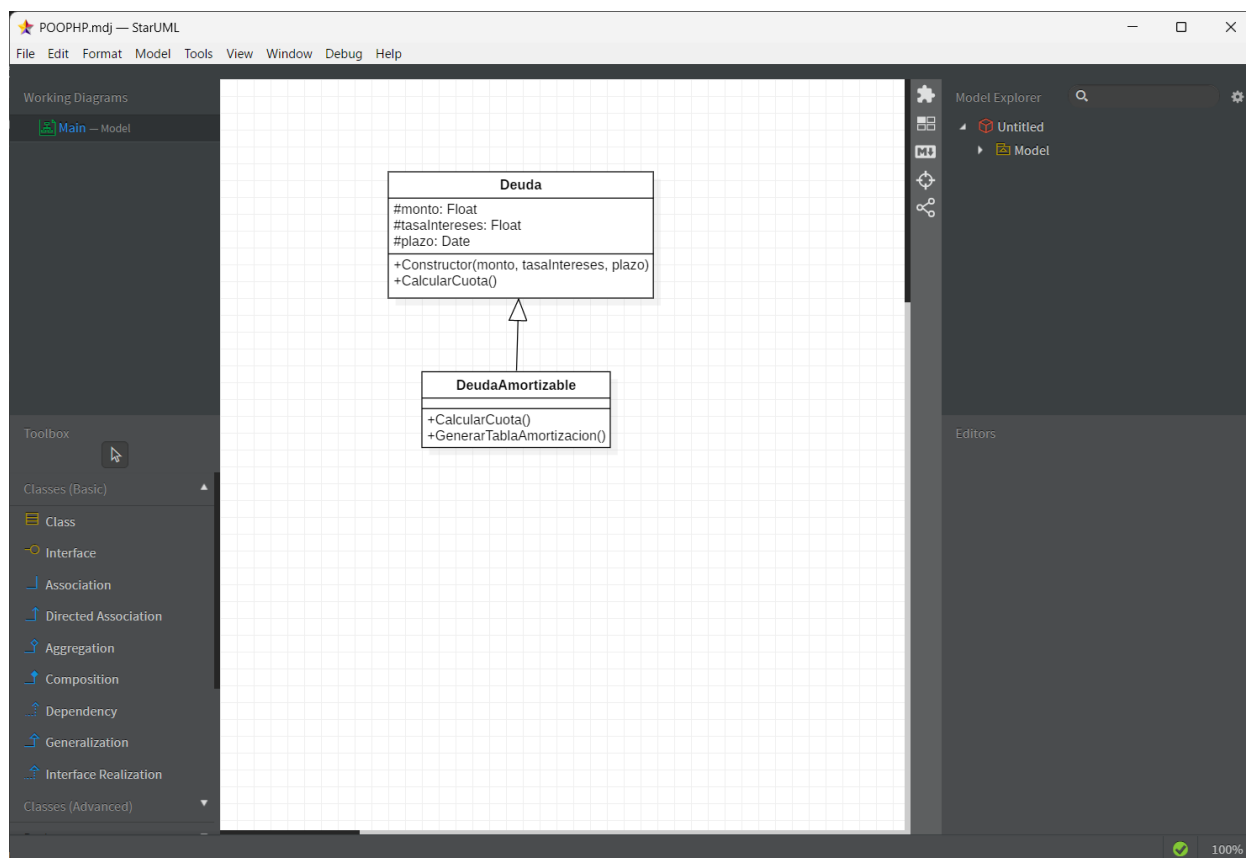
SENA Centro de Diseño y Metrología

Descripción del Proyecto

Este proyecto implementa un **simulador de tabla de amortización** desarrollado en **PHP** aplicando los principios de la **Programación Orientada a Objetos (POO)**.

Se diseñó un **diagrama UML** con dos clases principales:

- **Deuda:** clase base que contiene los atributos comunes como monto, tasa de interés y plazo.
- **DeudaAmortizable:** clase que hereda de Deuda y genera la tabla de amortización mensual con la cuota, interés, amortización y saldo.



El programa permite al usuario ingresar los datos del préstamo a través de un formulario web y genera automáticamente la tabla con todos los cálculos financieros. Además, incluye validaciones en el formulario y un diseño visual limpio y organizado.

Repositorio del proyecto

El código fuente completo se encuentra disponible en el siguiente enlace:

👉 <https://github.com/Jair-Arias/CLASE-NESTOR.git>

Dentro del repositorio podrás encontrar la carpeta **Segundo Trabajo** con el archivo **index.php**, que contiene todo el código necesario para ejecutar el simulador.

Cómo ponerlo a funcionar

1. **Instalar XAMPP** (si no lo tienes).
2. **Encender Apache** desde el panel de control de XAMPP.
3. Copiar la carpeta del proyecto en la ruta:

C:\xampp\htdocs\

Por ejemplo:

C:\xampp\htdocs\amortizacion\

4. Dentro de esa carpeta debe estar el archivo:

index.php

5. Abrir el navegador y acceder a:

http://localhost/amortizacion/

6. Ingresar los datos solicitados (monto, tasa y plazo) y presionar **Calcular**.

El sistema mostrará la **cuota mensual** y la **tabla de amortización** completa.

Conclusión

Este proyecto refleja el uso correcto de la **POO en PHP**, aplicando **herencia, encapsulamiento y reutilización de código**.

El desarrollo se apoyó en el diseño UML, que sirvió como base para la implementación de las clases y métodos en el sistema.

El resultado final es un simulador funcional, claro y adaptable, capaz de calcular cualquier deuda con su respectiva tabla de amortización.